



MANUAL DO ADMINISTRADOR (ROOT)

GOVERNANÇA, INFRAESTRUTURA & AUDITORIA DO SISTEMA

```
=====
Documento Oficial: Manual de Administração e Operação de TI (Root)
Versão: 2.0 (Edição de Governança & Segurança)
Público-Alvo: Administradores do Sistema, Equipe de TI e Engenharia de Software
Sistema: Portal de Tarefas Web + Backend Node.js + PostgreSQL + Bot Discord
Classificação: ESTRITAMENTE CONFIDENCIAL / ACESSO RESTRITO
=====
```



SUMÁRIO GERAL

1. Perfil e Princípios de Governança do ROOT
2. Autenticação Administrativa e Camada de Segurança
 - 2.1 PIN Reservado `000000` + Token Mestre
 - 2.2 Variáveis de Ambiente Críticas (`.env`)
3. Painel Geral de Tarefas (Supervisão Global)
 - 3.1 Isolamento Operacional do Root
 - 3.2 Visibilidade Completa sem Intervenção Indevida
4. Gerenciamento de Setores e Equipes
 - 4.1 Criação e Nomenclatura de Setores
 - 4.2 Edição e Desativação Segura de Setores
5. Gerenciamento de Usuários e Acessos
 - 5.1 Cadastro de Novo Colaborador e Geração de PIN
 - 5.2 Atribuição de Cargos e Setores
 - 5.3 Regeneração de PIN e Desvinculação do Discord
 - 5.4 Protocolo de Desativação e Desligamento de Colaborador
6. Central de Auditoria e Logs do Sistema
 - 6.1 Arquitetura dos Logs de Auditoria
 - 6.2 Filtros Avançados de Auditoria
 - 6.3 Investigação de Incidentes de Segurança e Erros
7. Terminal e Console SQL Interativo
 - 7.1 Diretrizes de Uso do Console SQL
 - 7.2 Regras de Execução e Travas de Segurança
 - 7.3 Exportação de Relatórios e Consultas Customizadas

- 8. Infraestrutura, Docker e Banco de Dados
 - 8.1 Estrutura dos Contêineres Docker
 - 8.2 Comandos de Manutenção e Restart
 - 8.3 Rotina de Backup e Restauração do PostgreSQL
 - 9. Matriz de Conformidade e Troubleshooting do Root
-

1. Perfil e Princípios de Governança do ROOT

O usuário **ROOT** é a autoridade máxima do sistema e representa a administração de TI e Segurança da Informação.

DIRETRIZES DE GOVERNANÇA DO ROOT

```
[ SUPERVISÃO TOTAL ] --> Visualiza todas as tarefas e logs
[ ISOLAMENTO ESTRITO ] --> Não recebe nem cria tarefas operacionais
[ CUSTÓDIA DE DADOS ] --> Gerencia usuários, equipes e dados brutos
[ AUDITORIA ATIVA ] --> Monitora conformidade e integridade
```

Princípios Inegociáveis:

1. **Neutralidade Operacional:** O Root nunca deve ser cadastrado como responsável ou criador de tarefas de rotina da empresa.
 2. **Imutabilidade de Histórico:** O Root não deve alterar dados de histórico de tarefas para mascarar auditoria.
 3. **Custódia das Credenciais:** O token mestre da `.env` nunca deve ser compartilhado por meios não criptografados.
-

2. Autenticação Administrativa e Camada de Segurança

2.1 PIN Reservado `000000` + Token Mestre

O acesso administrativo do Root não utiliza o fluxo padrão de colaboradores:

```

[ Usuário digita '000000' no campo de PIN do Portal ]
      |
      ▼
[ Sistema detecta requisição ROOT e abre Modal de Segurança ]
      |
      ▼
[ Administrador insere o TOKEN MESTRE secreto do servidor ]
      |
      ▼
[ Validação no Backend contra a variável ROOT_TOKEN do .env ]
      |
      ▼
[ Emissão de JWT Administrativo com Role: ROOT ]

```

Regras do Acesso Root:

- O PIN `000000` é **exclusivo e reservado**. Nenhum usuário comum pode receber esse PIN.
- A tentativa de login dispara um desafio de token administrativo.
- O token mestre é definido diretamente na variável de ambiente `ROOT_TOKEN`.
- O Root não depende de 2FA no Discord para entrar, garantindo acesso emergencial mesmo em caso de indisponibilidade da API do Discord.

2.2 Variáveis de Ambiente Críticas (`.env`)

No arquivo `.env` do backend, os seguintes parâmetros regem a segurança:

```

# Configurações do Root e JWT
JWT_SECRET=super_secret_jwt_key_gerado_aleatoriamente_com_alta_entropia
ROOT_TOKEN=ChaveMestraSecretaParaAcessoRoot2026!
PORT=3000

# Conexão com Banco de Dados
DB_HOST=postgres
DB_PORT=5432
DB_NAME=gestao_tarefas
DB_USER=postgres
DB_PASS=senha_forte_do_postgres

# Integração Discord
DISCORD_BOT_TOKEN=token_do_bot_obtido_no_discord_developer_portal
DISCORD_GUILD_ID=id_do_servidor_discord

```

3. Painel Geral de Tarefas (Supervisão Global)

3.1 Isolamento Operacional do Root

- O Root não possui uma coluna "Minhas Tarefas", pois não executa demandas operacionais.
- O Root tem acesso à visão global de **Todas as Tarefas** da organização.

3.2 Visibilidade Completa sem Intervenção Indevida

- O Root pode inspecionar qualquer card, ler comentários, verificar justificativas de pausa e auditar datas de entrega.
 - Por padrão de conformidade e governança, os botões de movimentação e conclusão de tarefas operacionais são desativados para a role Root, garantindo que apenas os colaboradores envolvidos respondam pelas entregas.
-

4. Gerenciamento de Setores e Equipes

Na aba **Gerenciar Equipes** (`/root/equipes`), o Root administra as unidades organizacionais da empresa.

4.1 Criação e Nomenclatura de Setores

1. Clique em **+ Nova Equipe**.
2. Defina o nome oficial do setor (ex: `Tecnologia da Informação`, `Financeiro`, `Comercial`, `Recursos Humanos`, `Operações`).
3. Clique em **Salvar**. O setor fica disponível imediatamente na listagem de criação de tarefas.

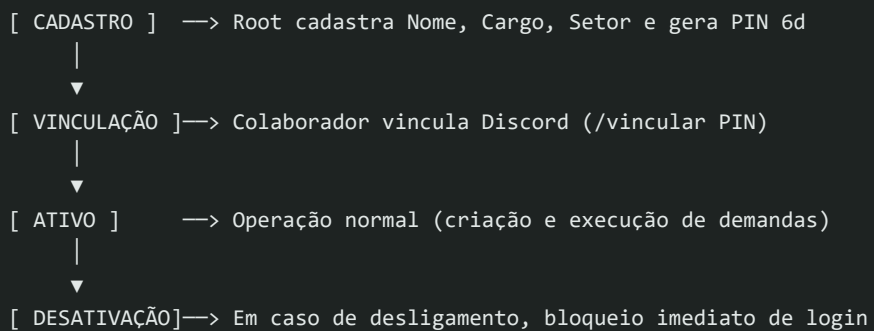
4.2 Edição e Desativação Segura de Setores

- **Renomear:** É possível alterar o nome de exibição de um setor sem quebrar o relacionamento com usuários e tarefas existentes (o `id` numérico permanece inalterado).
 - **Exclusão:** O sistema impede a exclusão de setores que possuam colaboradores ativos associados. Para excluir, é necessário reatribuir os colaboradores primeiro.
-

5. Gerenciamento de Usuários e Acessos

Na aba **Gerenciar Usuários** (`/root/usuarios`), o Root tem controle total do ciclo de vida dos colaboradores.

CICLO DE VIDA DO COLABORADOR NO PORTAL



5.1 Cadastro de Novo Colaborador e Geração de PIN

1. Clique em + **Novo Usuário**.
2. Preencha:
 - **Nome Completo**
 - **Setor** (selecionado da lista de setores)
 - **Cargo** (**FUNCIONARIO** , **COORDENADOR** , **GERENTE** , **DIRETOR**)
3. O sistema gera automaticamente um **PIN aleatório exclusivo de 6 dígitos**.
4. Entregue o PIN com segurança ao novo colaborador e oriente-o a realizar a vinculação no Discord.

5.2 Atribuição de Cargos e Setores

- O cargo define permissões especiais na interface:
 - **COORDENADOR** , **GERENTE** , **DIRETOR** : Têm acesso liberado à aba *Minha Equipe*.
 - **FUNCIONARIO** : Foco estrito em *Minhas Tarefas e Criadas por Mim*.

5.3 Regeneração de PIN e Desvinculação do Discord

Em caso de esquecimento de PIN, troca de conta do Discord ou suspeita de vazamento de credencial:

1. Localize o usuário na lista.
2. Clique em **Gerar Novo PIN / Resetar Vínculo**.
3. O vínculo anterior do Discord é revogado no banco de dados (`discord_id = NULL`).
4. Um novo PIN é emitido e o usuário deverá executar `/vincular NOVO_PIN` novamente no Discord.

5.4 Protocolo de Desativação e Desligamento de Colaborador

Ao desligar um colaborador:

1. Localize o usuário e clique no toggle **Desativar**.
2. O sistema revoga tokens JWT ativos instantaneamente.

3. O usuário inativo não conseguirá mais efetuar login e não aparecerá como opção para receber novas tarefas.
4. **Histórico Preservado:** Todas as tarefas anteriormente criadas ou concluídas pelo usuário permanecem intactas no histórico corporativo.

6. Central de Auditoria e Logs do Sistema

A aba **Logs de Auditoria** (`/root/logs`) armazena permanentemente cada interação no sistema.

6.1 Arquitetura dos Logs de Auditoria

Cada registro de log armazena:

- `id` : Identificador único sequencial.
- `created_at` : Data e hora com precisão de milissegundos.
- `usuario_id` / `usuario_nome` : Quem executou a ação.
- `acao` : Verbo da operação (`LOGIN` , `CREATE_TASK` , `UPDATE_STATUS` , `ADD_COMMENT` , `USER_CREATE` , `USER_UPDATE` , etc.).
- `entidade` : Objeto afetado (`TAREFA` , `USUARIO` , `SETOR` , `AUTH` , `SQL`).
- `entidade_id` : ID da entidade modificada.
- `detalhes` : Objeto JSON com dados anteriores e novos (diff).
- `ip_address` : Endereço IP de origem da requisição.

6.2 Filtros Avançados de Auditoria

A tela dispõe de ferramentas de filtragem em tempo real:

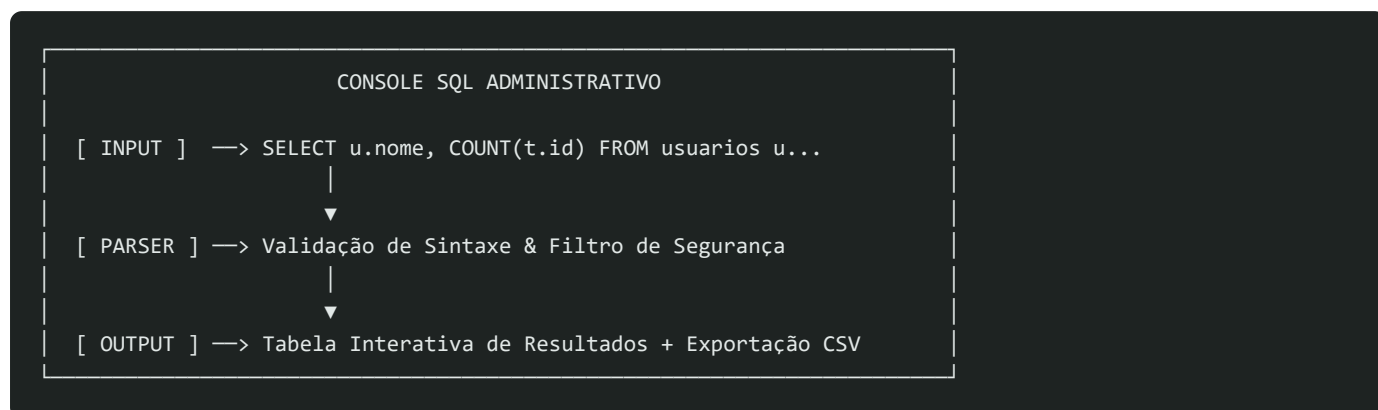
- **Busca por Termo:** Localiza por nome de usuário, IP ou palavras-chave nos detalhes.
- **Filtro por Ação:** Exibe apenas logins, mudanças de status, edições, etc.
- **Filtro por Entidade:** Isola logs de Tarefas, Usuários ou Segurança.
- **Filtro por Intervalo de Data:** Auditoria por período específico.
- **Exportação CSV:** Exportação do conjunto filtrado para relatórios de compliance.

6.3 Investigação de Incidentes de Segurança e Erros

- **Tentativas de Login com PIN Inválido:** Registradas com alerta de segurança e IP de origem.
 - **Transições Proibidas:** Tentativas de burlar regras de movimentação de tarefas são barradas no backend e logadas.
 - **Auditoria de Consultas SQL:** Toda query executada no Console SQL fica gravada com o comando exato.
-

7. Terminal e Console SQL Interativo

Disponível no menu **Console SQL** (`/root/sql`), permite à equipe de TI realizar consultas diretas e diagnósticos sem necessidade de ferramentas externas.



7.1 Diretrizes de Uso do Console SQL

- O console é destinado para **consultas analíticas** (`SELECT`), diagnósticos e extração rápida de métricas gerenciais.
- Consultas são executadas em pool seguro com limite automático de linhas para não comprometer a memória do servidor.

7.2 Regras de Execução e Travas de Segurança

- O sistema mantém um **Histórico de Consultas** executadas recentemente para reutilização ágil.
- Permite formatar queries SQL com um clique.

7.3 Exportação de Relatórios e Consultas Customizadas

- Qualquer resultado gerado na grade pode ser baixado instantaneamente em formato `.CSV` para consolidação em Excel ou ferramentas de BI.

8. Infraestrutura, Docker e Banco de Dados

8.1 Estrutura dos Contêineres Docker

O ambiente é orquestrado via `docker-compose.yml` :

Serviço	Contêiner	Porta Interna	Porta Externa	Função
Frontend	portal_frontend	80 / 5173	5173 / 80	Aplicação SPA React (Vite / Nginx)
Backend	portal_backend	3000	3000	API RESTful Node.js + Express
Banco de Dados	portal_postgres	5432	5432	PostgreSQL 16 com persistência em volume
Bot Discord	portal_bot	-	-	Worker Node.js conectado à API do Discord

8.2 Comandos de Manutenção e Restart

Executar no diretório raiz do projeto:

```
# Subir toda a infraestrutura em segundo plano
docker compose up -d

# Visualizar logs em tempo real de todos os serviços
docker compose logs -f

# Reiniciar apenas o backend após alteração de código/.env
docker compose restart backend

# Reiniciar o bot do Discord
docker compose restart bot

# Parar todos os serviços
docker compose down
```

8.3 Rotina de Backup e Restauração do PostgreSQL

Backup Manual (Dump SQL):

```
# Gerar backup compactado do banco de dados
docker compose exec postgres pg_dump -U postgres gestao_tarefas > backup_tarefas_$(Get-Date -Format 'yyyyMMdd')
```

Restauração do Banco de Dados:

```
# Restaurar backup a partir de arquivo SQL
Get-Content backup_tarefas_20260831.sql | docker compose exec -T postgres psql -U postgres gestao_tarefas
```


9. Matriz de Conformidade e Troubleshooting do Root

Sintoma / Incidente	Causa Provável	Ação Corretiva do Root
Bot Discord não envia notificações	<code>DISCORD_BOT_TOKEN</code> expirado ou bot sem permissão de DM	Verificar status do bot no Discord Developer Portal, atualizar <code>.env</code> e executar <code>docker compose restart bot</code> .
Login do Root retorna "Token Inválido"	<code>ROOT_TOKEN</code> informado difere do <code>.env</code>	Conferir a variável <code>ROOT_TOKEN</code> no arquivo <code>.env</code> do backend.
Usuário não recebe código de login	Discord do usuário não vinculado ou DMs bloqueadas	Verificar na tela de Usuários se o campo <code>discord_id</code> está preenchido. Se necessário, resetar o PIN.
Erro de conexão com o banco (<code>ECONNREFUSED</code>)	Contêiner PostgreSQL parado ou inicializando	Executar <code>docker compose ps</code> e <code>docker compose restart postgres</code> .

Manual restrito à equipe de TI e Administradores do Sistema.